Introduction to Containers

Robin Langer and Claude Code

July 2026

1 Containers and Types

Definition 1.1. A *container* is a pair $\psi = (S, P)$ where S is a set of *shapes* and

$$P : S \to \mathbf{Set}(A)$$

is a family of *positions*, where $\mathbf{Set}(A)$ is the set of all subsets of A for some universal set A.

For any given shape s, think of $P(s) \subseteq A$ as a set of "slots" where you can put data. We don't actually have any data yet. That will come later when we do extensions of containers.

Example 1.2. A *list* is described by the container L with shapes $S_L = \mathbb{N}$ (the length) and positions

$$P_L : \mathbb{N} \to \mathbf{Set}(\mathbb{N}), \qquad P_L(n) = \{1, 2, \ldots, n\}.$$

Note that this container represents lists of any length. But it doesn't yet contain any data. Think of it as the type of a list.

Example 1.3. Encode a vertex of a binary tree as a word in $\{\mathrm{L}, \mathrm{R}\}^*$ — the directions taken from the root. A finite set $t \subseteq \{\mathrm{L}, \mathrm{R}\}^*$ is a *valid tree shape* when it is *prefix-closed*. That is, every prefix w' of a word $w \in t$ is also in t. The empty word (the root) belongs to every nonempty tree. Let Tree be the set of all valid tree shapes, and let

$$T = (\mathrm{Tree}, Q), \qquad Q : \mathrm{Tree} \to \mathbf{Set}(\{\mathrm{L}, \mathrm{R}\}^*), \qquad Q(t) = t,$$

We denote by $V(t)$ the number of vertices of t.

If you already have an image in your mind of what a tree looks like, imagine that each vertex of the tree is a "slot" where we could add data. As before, we don't have any data yet. This is just the type of a tree.

Remark 1.4. For any container (S, P) and any $s \in S$ we may associate a *dependent type*:

$$s : S \vdash P(s) : \texttt{Type}.$$

Very few programming languages support dependent types, the exceptions being Lean, Agda, Idris, and Coq.

We haven't actually said what a `Type` is yet. But we will in Section 4.

Example 1.5. Haskell does not support dependent types. In Haskell the list and the tree are defined by

```
data List a = Nil | Cons a (List a)
data Tree a = Leaf | Node (Tree a) a (Tree a)
```

A list of `a`s is either empty, or an element together with another list. A tree is either a leaf, or a node carrying an element and two subtrees. The types are defined recursively.

In Haskell we cannot define a list of a specific length or a tree of a specific shape. The best we can do is a run-time check.

```
mkListOfLength :: Int -> [a] -> Maybe [a]
mkListOfLength n xs
   | length xs == n = Just xs
   | otherwise      = Nothing
```

Example 1.6. Recursion in C is implemented using pointers. A position is an *address*. A labelling of the positions with data is the *memory contents*.

What is the type of a *linked list*? In C one writes

```
typedef struct Node *List;

struct Node {
   int data;
   struct Node *next;
};
```

The list is given the type of a pointer to its first node. In C, every pointer type is either a valid address or `NULL`. Unfolding it, a node is a datum together with either another node or `NULL`, so a linked list is either empty, or a head and a tail. C cannot express the recursion directly, so it smuggles it in with pointers.

Likewise a binary tree.

```
typedef struct TNode *Tree;

struct TNode {
   int data;
   struct TNode *left;
   struct TNode *right;
};
```

A tree is a pointer to its root, the empty tree is `NULL`, and each node now carries two pointers — one slot for each child.

What does C check? Pointer types are checked at compile time, a pointer to one type of struct cannot point to a struct of a different type, and within a struct only the declared fields can be named.

What does C not check?

- Bounds — trying to access the 7th element of a list of length 5 (the buffer overflow).
- Nullness — following a pointer that points to `NULL`.
- Dangling pointers — for example, after removing an element from the end of a list you might forget to point the previous pointer to `NULL`.

This is ugly, which is the reason why people prefer to program in higher level languages.

Example 1.7. One language that does support dependent types is Lean. For example, we can define a vector of length `n` containing entries from `a` as follows:

```
inductive Vec (a : Type) : Nat -> Type where
   | nil  : Vec a 0
   | cons : a -> Vec a n -> Vec a (n + 1)
```

`Vec a n` is the type of a list of length `n`. Note that it doesn't actually have any data yet. It is a type.

Note that `(Vec a)` is a *type constructor*, of signature

```
(Vec a) : Nat -> Type
```

The type constructor depends on a *value* `n` of type `Nat`, and returns a `Type`.

A function from values to types is called a *dependent family* — it is the same kind of object as the position family $P_L : \mathbb{N} \to \mathbf{Set}(\mathbb{N})$ that we saw for lists.

$P_L(n) = \{1, 2, \ldots, n\}$ is `Vec a n`.

To construct a vector of length 3 in Lean we write:

```
def v : Vec Int 3 := cons 1 (cons 2 (cons 3 nil))
```

The list now contains data, `v` is a value of type `Vec Int 3`. This is a recursive construction. The length is tracked in the type at every step. The way we put data into containers is different. We don't need recursion, instead we just "slot it in". This is one of the great powers of container theory.

Why would anybody do this? The idea is to catch a vector length mis-match at compile time rather than run time.

Dependent types come at a price. If types contain values, then deciding whether two types are equal requires deciding whether two values are equal, so the type checker must *execute code* at compile time.

If types contain values, then types contain *computations*. For example:

```
Vec a (n + m) = Vec a (m + n)
```

describe vectors of the same length, but we must prove they are the same type. This is extremely tedious and painful.

2 Sums and products of containers

Definition 2.1. The *categorical sum* of two sets A and B is their *disjoint union* $A \sqcup B$.

$$\text{left} : A \to A \sqcup B, \qquad \text{right} : B \to A \sqcup B.$$

The sum extends to *functions*. Given $f : A \to C$ and $g : B \to D$, their sum

$$f \sqcup g : A \sqcup B \to C \sqcup D$$

is defined by the unique function making the two squares commute.

$$\begin{array}{ccc} A & \xrightarrow{\text{left}} & A \sqcup B \\ \downarrow f & & \downarrow f \sqcup g \\ C & \xrightarrow{\text{left}} & C \sqcup D \end{array} \qquad \begin{array}{ccc} B & \xrightarrow{\text{right}} & A \sqcup B \\ \downarrow g & & \downarrow f \sqcup g \\ D & \xrightarrow{\text{right}} & C \sqcup D \end{array}$$

Definition 2.2. The *categorical product* of two sets A and B is their *cartesian product* $A \times B$. It comes with the two projections

$$\pi_1 : A \times B \to A, \qquad \pi_2 : A \times B \to B.$$

The product extends to *functions*. Given $f : A \to C$ and $g : B \to D$, their product

$$f \times g : A \times B \to C \times D$$

is the unique function making the two squares commute.

$$\begin{array}{ccc} A \times B & \xrightarrow{\pi_1} & A \\ \downarrow f \times g & & \downarrow f \\ C \times D & \xrightarrow{\pi_1} & C \end{array} \qquad \begin{array}{ccc} A \times B & \xrightarrow{\pi_2} & B \\ \downarrow f \times g & & \downarrow g \\ C \times D & \xrightarrow{\pi_2} & D \end{array}$$

Since we used universal properties as definitions, sums and products can be defined for any category and they are dual. Both sum and product "lift" naturally to containers.

Definition 2.3. Suppose that we have two containers $\psi = (S, P)$ with $P : S \to \mathbf{Set}(A)$ and $\varphi = (T, Q)$ with $Q : T \to \mathbf{Set}(B)$. Their *sum* $\psi + \varphi$ is the container whose shapes are the disjoint union $S \sqcup T$ and whose positions are

$$(P + Q) : S \sqcup T \to \mathbf{Set}(A \sqcup B), \qquad (P + Q)(s) = \begin{cases} P(s) & \text{if } s \in S, \\ Q(s) & \text{if } s \in T. \end{cases}$$

The *product* $\psi \times \varphi$ is the container whose shapes are the pairs $S \times T$ and whose positions are the disjoint union

$$(P \times Q) : S \times T \to \mathbf{Set}(A \sqcup B), \qquad (P \times Q)(s, t) = P(s) \sqcup Q(t).$$

Example 2.4. Recall the list container $L = (\mathbb{N}, P_L)$ with $P_L : \mathbb{N} \to \mathbf{Set}(\mathbb{N})$, $P_L(n) = \{1, 2, \ldots, n\}$. Note that

$$\mathbb{N} = \{1\} \cup \{2\} \cup \{3\} \cup \cdots$$

so the shapes decompose as the disjoint union of the singletons $S_k = \{k\}$. Define a family of individual shape containers. Let $P_k : \{k\} \to \mathbf{Set}(\mathbb{N})$ with $P_k(k) = \{1, 2, \ldots, k\}$, and set $L_k = (S_k, P_k)$. Then

$$L = L_1 + L_2 + L_3 + \cdots$$

Each L_k is a dependent type — it is `Vec a k`, whose type depends on the value k — but the sum L is not. It is the ordinary type `List a`, which we can have in Haskell.

Example 2.5. Recall the tree container $T = (\text{Tree}, Q)$ of Example 1.3, with $Q(t) = t$. Now Tree decomposes as

$$\text{Tree} = \bigcup_{t \in \text{Tree}} \{t\},$$

and for each t we have the individual shape container $T_t = (\{t\}, Q_t)$ with $Q_t(t) = t$. So

$$T = \sum_{t \in \text{Tree}} T_t,$$

Again each T_t is a dependent type — the type of trees of valid tree word exactly t, which depends on the value t — but the sum T is not. It is the ordinary type `Tree a`, which we can have in Haskell.

Every container is the sum of its individual shape subcontainers. $\psi = \sum_{s \in S}(\{s\}, P_s)$.

3 Container Morphisms

Definition 3.1. Suppose that we have two containers $\psi = (S, P)$ with

$$P : S \to \mathbf{Set}(A)$$

and $\varphi = (T, Q)$ with

$$Q : T \to \mathbf{Set}(A).$$

They must have the same set of possible positions A. A *morphism* of containers $(u, g) : \psi \to \varphi$ is a map from the shapes S to the shapes T

$$u : S \to T$$

together with a family of arrows, one for each shape $s \in S$, pointing in the *opposite* direction:

$$g_s : Q(u(s)) \mapsto P(s).$$

The new positions of T can only come from the old positions of S. This is because new data cannot be created. The arrow points backwards to tell you where the data in $t \in T$ came from.

Example 3.2. reverse is the morphism $L \to L$ of the list container with

$$u : n \mapsto n \qquad \text{and} \qquad g_n : k \mapsto (n + 1 - k).$$

This says that the new position k came from the old position $(n + 1 - k)$.

Example 3.3. Let $\text{dup} : L \to L$ be the operation sending the list $[a, b, c]$ to $[a, a, b, b, c, c]$. As a container morphism the shape arrow doubles the length,

$$u : n \mapsto 2n,$$

and the positions arrow sends each new position back to the old position it copies its data from,

$$g_n : (2k - 1) \mapsto k \qquad \text{and} \qquad g_n : 2k \mapsto k.$$

This tells you that both the new positions 3 and 4 came from the old position 2.

Example 3.4. Let tail : $L \to L$ be the operation sending the list $[a, b, c, d]$ to $[b, c, d]$. As a container morphism the shape arrow decreases the length by one,

$$u : n \mapsto n - 1,$$

and the positions arrow is

$$g_n : k \mapsto (k + 1).$$

This says that the new position k came from the old position $(k + 1)$.

Theorem 3.5. *Containers and container morphisms form a category* **Cont**. *The identity morphism on $\psi = (S, P)$ is*

$$\mathrm{id}_\psi = \big(\mathrm{id}_S,\ (\mathrm{id}_{P(s)})_{s \in S}\big),$$

where for each $s \in S$, the map $\mathrm{id}_{P(s)}$ is the identity function. Composition is given by:

$$(v, h) \circ (u, g) = \big(v \circ u,\ (g_s \circ h_{u(s)})_{s \in S}\big).$$

The shape maps compose forwards,

$$S \xrightarrow{u} T \xrightarrow{v} \tilde{S}$$

while the position arrows compose backwards

$$P(s) \xleftarrow{g_s} Q(u(s)) \xleftarrow{h_{u(s)}} \tilde{P}(v(u(s))).$$

Remember that we can't create new data, so the position arrows tell us where the data came from. The data doesn't live in T, it lives in $Q(t)$ for some $t \in T$. The composite $g_s \circ h_{u(s)}$ only has the right type because the arrow on positions points backwards.

Associativity and the identity laws are inherited from ordinary function composition.

Remark 3.6. Every container morphism $f : A \to B$ can be thought of as a function of type $A \to B$, but not every function of type $A \to B$ is a container morphism.

For example, permuting a list is a container morphism but sorting a list is not.

For simplicity, suppose our language supports only the primitives `Int` and `Bool`, together with their basic operations. These are arithmetic, comparison and boolean operations. None of these basic operations is a container morphism.

To get a complete language we must also add *pattern matching* (branching) and *recursion*. Pattern matching is case analysis on the constructor of a value.

Example 3.7. Consider *tree sort*. The type of this map is sort : $L \times T \to L \times T$. First we insert the elements from the list into the tree (according to some criterion), then we read back the elements from the tree to the list (in a certain order). The state at each step is a pair (L, T). It records what is left to insert, and what has been built so far — a list of length n and a search tree t with k vertices. One step of the algorithm takes the head of the list and inserts it into the tree:

$$\mathrm{insert} : L \times T \to L \times T, \qquad (n, t) \longmapsto (n - 1,\ t'),$$

where t' has one vertex more than t, and for the read-back phase

$$\mathrm{read} : L \times T \to L \times T, \qquad (n, t) \longmapsto (n + 1,\ t''),$$

where t'' has one less vertex than t.

For $n = 3$ the full sort algorithm can be expressed as

$$\text{read} \circ \text{read} \circ \text{read} \circ \text{insert} \circ \text{insert} \circ \text{insert}.$$

In general, define the iterates recursively. Set

$$\text{insert}_1 = \text{insert}, \qquad \text{insert}_{n+1} = \text{insert} \circ \text{insert}_n,$$

and likewise

$$\text{read}_1 = \text{read}, \qquad \text{read}_{n+1} = \text{read} \circ \text{read}_n.$$

This allows us to apply insertion sort to a list of any length n.

$$\text{sort}_n = \text{read}_n \circ \text{insert}_n.$$

In Haskell:

```
insert :: Ord a => ([a], Tree a) -> ([a], Tree a)
insert (x : xs, t) = (xs, ins x t)
   where
      ins x Leaf = Node Leaf x Leaf
      ins x (Node l y r)
         | x <= y   = Node (ins x l) y r
         | otherwise = Node l y (ins x r)

read :: ([a], Tree a) -> ([a], Tree a)
read (xs, t) = (y : xs, t')
   where
      (y, t') = removeMax t
      removeMax (Node l x Leaf) = (x, l)
      removeMax (Node l x r)  = (z, Node l x r')
         where (z, r') = removeMax r

insertN :: Ord a => Int -> ([a], Tree a) -> ([a], Tree a)
insertN 1 = insert
insertN n = insert . insertN (n - 1)

readN :: Int -> ([a], Tree a) -> ([a], Tree a)
readN 1 = read
readN n = read . readN (n - 1)

sortN :: Ord a => Int -> ([a], Tree a) -> ([a], Tree a)
sortN n = readN n . insertN n
```

This is NOT a container morphism because it uses comparisons, and pattern matching, and recursion.

4 Extension of containers and Types

Now we are ready to put some data into our containers. In this section we will "bootstrap" the definition of a type.

Definition 4.1. The *extension* of a container $\psi = (S, P)$ with respect to a set of labels X (data) is

$$C_\psi(X) = \sum_{s:S} \big(P(s) \to X\big) = \big\{\langle s, f\rangle : s \in S,\ f : P(s) \to X\big\}.$$

The elements of the extension are pairs $\langle s, f\rangle$. A shape s together with a labelling f of its positions. Think of it as slotting data into the positions of $P(s)$. For example, if $X = \{\text{True}, \text{False}\}$, then $C_L(X)$ is the set of lists of type `Bool` of any length.

You can think of the container as "pure structure" and the variable X as a set of labels on that structure. The labels are data that can be "slotted in" to the positions.

Remark 4.2. A container morphism $(u, g) : \psi \to \varphi$ acts on extensions by

$$\langle s, f\rangle \longmapsto \langle u(s),\ f \circ g_s\rangle.$$

Note again that the arrow g_s points backwards, otherwise you wouldn't be able to compose it with f. We have $f : P(s) \to X$ and $g_s : \hat{P}(u(s)) \mapsto P(s)$, so $f \circ g_s : \hat{P}(u(s)) \to X$ is a labelling of the new shape.

We are going to "bootstrap" the definition of `Type` by setting $\texttt{Type}_0 = \{\texttt{Int}, \texttt{Bool}\}$ and then defining $\texttt{Type}_{k+1}$ recursively in terms of $\texttt{Type}_k$.

You can think of $\texttt{Type}_3$ as types of types of types of $\{\texttt{Int}, \texttt{Bool}\}$.

Let us define a 2-indexed container first, we'll work up to the general definition later. Start with a "struct" containing three `Int`s and two `Bool`s.

```
struct {
   int x, y, z;
   bool a, b;
}
```

As a container this is a single shape s with five slots

$$P(s) = \{x, y, z, a, b\}.$$

The slots come in two "types". We record the type of each slot with a map into the index set $I = \{\texttt{Int}, \texttt{Bool}\}$,

$$r_s : P(s) \to I,$$

so $r_s(x) = r_s(y) = r_s(z) = \texttt{Int}$ and $r_s(a) = r_s(b) = \texttt{Bool}$.

Definition 4.3. Let I be an index set with two elements. A 2-*indexed container over* I is a container $\phi = (S, P)$ with, for each $s \in S$, a map

$$r_s : P(s) \to I,$$

called the *response map*.

Take $I = \{\texttt{Int}, \texttt{Bool}\}$. To put data into the container we choose one data set for each index.

$$X_1 = \mathbb{N} \qquad \text{for } \texttt{Int}, \text{ and} \qquad X_2 = \{\text{True}, \text{False}\} \qquad \text{for } \texttt{Bool}.$$

Definition 4.4. The *extension* of a 2-indexed container $\phi = (S, P, r)$ over I at the data (X_1, X_2) is

$$C_\phi(X_1, X_2) = \sum_{s \in S} \prod_{p \in P(s)} X_{r_s(p)},$$

where $X_{\texttt{Int}} = X_1$ and $X_{\texttt{Bool}} = X_2$.

Compare this with the plain extension $C_\psi(X)$ of Definition 4.1. There every slot received data from the *same* set X, and a labelling was a function $f : P(s) \to X$. Here each slot p has its own data type $X_{r_s(p)}$. An `Int` slot receives a value from X_1 and a `Bool` slot receives a value from X_2.

For the struct above,

$$C_\phi(X_1, X_2) = \mathbb{N}^3 \times \{\text{True}, \text{False}\}^2.$$

An element of the extension slots an actual value into each field. We now have data in our containers.

Now we give the general definition. The index set I can be any set, the data is a family $X = (X_i)_{i \in I}$ with one set for each index, and the shapes are typed as well as the positions.

Definition 4.5. Let I be any index set. An *I-indexed container* is a container $\phi = (S, P)$ with a map $r : S \to I$ typing the shapes and, for each $s \in S$, a response map $r_s : P(s) \to I$ typing the positions. Given data $X = (X_i)_{i \in I}$, the *extension* of ϕ at $i \in I$ is obtained by restricting to $S_i = \{\, s \in S \mid r(s) = i \,\}$, the shapes of type i:

$$C_{\phi,i}(X) = \sum_{s \in S_i} \prod_{p \in P(s)} X_{r_s(p)}.$$

Think of I as dividing up the positions into different "types". If p is of type i then we choose a value from the data set X_i.

We can bootstrap the definition of `Type` using indexed container extensions.

$$\begin{aligned}
\texttt{Type}_0 &= \{\texttt{Int}, \texttt{Bool}\},\\
\texttt{Type}_1 &= \text{indexed container extensions over } \texttt{Type}_0,\\
\texttt{Type}_2 &= \text{indexed container extensions over } \texttt{Type}_1,\\
&\vdots\\
\texttt{Type}_{k+1} &= \text{indexed container extensions over } \texttt{Type}_k.
\end{aligned}$$

Definition 4.6. Define, by induction on the *depth* k,

$$\texttt{Type}_0 = T = \{\texttt{Int}, \texttt{Bool}\},$$

and $\texttt{Type}_{k+1}$ to be $\texttt{Type}_k$ together with the extensions $C_{\phi,x}$ of indexed containers ϕ over $\texttt{Type}_k$. The universe of types is

$$\texttt{Type} = \bigcup_{k \geq 0} \texttt{Type}_k,$$

and every type has a finite depth.

5 Values from types

We have defined the types. To get the *values* we perform one last container extension. Take the indexed container over $I = \texttt{Type}_0$ with shapes $S = \{\texttt{Int}, \texttt{Bool}\}$, where each type has a single slot of that type. The data is $X_{\texttt{Int}} = \mathbb{N}$ and $X_{\texttt{Bool}} = \{\text{True}, \text{False}\}$. The extension is

$$C = \sum_{i \in \texttt{Type}_0} X_i = \mathbb{N} \sqcup \{\text{True}, \text{False}\} = A,$$

the set of values. An element of the extension is a pair of a type and a value of that type. This final extension turns a type into a value.

Note that $\mathbb{N}$ itself is a container. Take two shapes, zero and succ, where zero has no slots and succ has exactly one.

$$S = \{\text{zero}, \text{succ}\}, \qquad P(\text{zero}) = \varnothing, \qquad P(\text{succ}) = \{*\}.$$

A natural number is either zero, or succ with another natural number slotted into its single slot.

$$3 = \text{succ}(\text{succ}(\text{succ}(\text{zero})))$$

The naturals are what you get by slotting this container into itself, over and over.

Likewise `Bool` is a container. Take two shapes, true and false, neither of which has any slots.

$$S = \{\text{true}, \text{false}\}, \qquad P(\text{true}) = P(\text{false}) = \varnothing.$$

A boolean is a bare shape with nothing slotted in. There is no recursion this time. The two shapes have the same (empty) position set, so `Bool` is non-dependent.

6 De-categorization

Remark 6.1. One can show that for container extensions:

$$C_{\psi+\varphi}(X) = C_\psi(X) + C_\varphi(X), \qquad C_{\psi\times\varphi}(X) = C_\psi(X) \times C_\varphi(X).$$

Remark 6.2. Replacing sets by their cardinalities turns a container into a formal power series. This is called *de-categorification*. Imagine that X is a set with a single element x, so we are just counting the pure structure of the container, not the data. Concretely, the set of functions $(P(s) \to X)$ can also be written as the exponential $X^{P(s)}$, so the extension is

$$C_\psi(X) = \sum_s X^{P(s)}.$$

Now let a_n be the number of shapes with n positions. Then replacing each set by its cardinality and gathering together powers of n we get the de-categorification of ψ.

$$f(x) = \sum_n a_n x^n = a_0 + a_1 x + a_2 x^2 + \cdots.$$

For the list container there is exactly one shape with n positions, so

$$f(x) = \sum_n x^n = \frac{1}{1-x}.$$

For the tree container T the number of shapes with n vertices is the Catalan number

$$C_n = \frac{1}{n+1}\binom{2n}{n},$$

so

$$f(x) = \sum_n C_n x^n = \frac{1-\sqrt{1-4x}}{2x} = 1 + x + 2x^2 + 5x^3 + 14x^4 + \cdots$$

This satisfies $f(x) = 1 + xf(x)^2$. A tree is empty, or a root vertex with two subtrees.

6.1 Tensor Products

Definition 6.3. Suppose that we have two containers $\psi = (S, P)$ with $P : S \to \mathbf{Set}(A)$ and $\varphi = (\hat{S}, \hat{P})$ with $\hat{P} : \hat{S} \to \mathbf{Set}(\hat{A})$. Their *tensor product* $\psi \otimes \varphi$ is the container whose shapes are the pairs $S \times \hat{S}$ and whose positions are the pairs

$$(P \otimes \hat{P}) : S \times \hat{S} \to \mathbf{Set}(A \times \hat{A}), \qquad (P \otimes \hat{P})(s, \hat{s}) = P(s) \times \hat{P}(\hat{s}).$$

A shape of the tensor product is a shape from each container, and its "slots" form a grid. One slot for every pair of a position of s and a position of $\hat{s}$. The extension is

$$C_{\psi\otimes\varphi}(X) = \sum_{(s,\hat{s})} \big(P(s) \times \hat{P}(\hat{s}) \to X\big).$$

Unlike the sum, product and composition, this cannot be expressed in terms of $C_\psi(X)$ and $C_\varphi(X)$ alone.

Example 6.4. Let L be the list container with shapes $S_L = \mathbb{N}$ and positions $P_L(n) = \{1, \ldots, n\}$. The tensor product $L \otimes L$ has shapes $\mathbb{N} \times \mathbb{N}$ and positions

$$(P_L \otimes P_L)(m, n) = \{1, \ldots, m\} \times \{1, \ldots, n\}.$$

The extension is

$$C_{L\otimes L}(X) = \sum_{(m,n)} \big(\{1, \ldots, m\} \times \{1, \ldots, n\} \to X\big),$$

which is the set of $m \times n$ *matrices* with entries in X, of any size. A list tensor a list is a matrix.

Remark 6.5. For $L \otimes L$ the de-categorification gives the Lambert series

$$h(x) = \sum_{j\geq 1} \frac{x^j}{1-x^j} = \sum_{n\geq 1} d(n)\, x^n,$$

where $d(n)$ is the number of divisors of n: a matrix with 6 entries can be 1×6, 2×3, 3×2 or 6×1.

Example 6.6. The tensor product $L \otimes T$ has shapes $\mathbb{N} \times \mathrm{Tree}$ and positions

$$(P_L \otimes Q)(n, t) = \{1, \ldots, n\} \times t.$$

A shape is a length n together with a tree t, and there is one slot for every pair of a list position and a vertex of t. So an element of the extension $C_{L\otimes T}(X)$ is a tree in which every vertex holds a list of length n — the *same* length n at every vertex. Equivalently, it is a list of n trees that all have the same shape t.

Example 6.7. The tensor product $T \otimes T$ has shapes Tree $\times$ Tree and positions

$$(Q \otimes Q)(t, t') = t \times t' \subseteq \{\mathrm{L}, \mathrm{R}\}^* \times \{\mathrm{L}, \mathrm{R}\}^*.$$

A shape is a pair of trees (t, t'), and there is one slot for every pair of a vertex of t and a vertex of t'. So an element of the extension $C_{T\otimes T}(X)$ is a tree t in which every vertex holds a copy of the *same* tree shape t', labelled by X — or equivalently a tree of shape t' in which every vertex holds a copy of the tree shape t. Whatever tree you replace the vertices with, they must all be the same shape.

6.2 Composition of containers

Definition 6.8. The *composition* $\psi \circ \varphi$ is the container whose shapes are the elements of the extension $C_\psi(\hat{S})$ — a shape s of ψ with a shape of φ slotted into each of its positions — and whose positions, for a shape $\langle s, f\rangle$ with $f : P(s) \to \hat{S}$, are the pairs

$$(P \circ \hat{P})\langle s, f\rangle = \{\, (p, \hat{p}) \mid p \in P(s),\ \hat{p} \in \hat{P}(f(p)) \,\} \subseteq A \times \hat{A}.$$

Let us unpack this definition. A shape $\langle s, f\rangle$ is a shape s of ψ together with a function f that slots a shape of φ into each position of s. A position of the composite is a pair $(p, \hat{p})$. First pick a position $p \in P(s)$ (an element of A, since $P(s) \subseteq A$), then look up the inner shape $f(p)$ sitting at p and pick one of *its* positions $\hat{p} \in \hat{P}(f(p))$ (an element of $\hat{A}$). So a slot of the composite is a slot of the outer structure followed by a slot of the inner structure found there.

For example, $T \circ T$ is the container of trees of trees. A shape is a tree t together with a tree shape for each vertex of t. Unlike the tensor product $T \otimes T$, the inner trees may all have *different* shapes.

Definition 6.9. The extension of the composition is the composite of the two extensions:

$$C_{\psi\circ\varphi}(X) = C_\psi(C_\varphi(X)).$$

Let us unpack this definition. An element of the right-hand side $C_\psi(C_\varphi(X))$ is a ψ-structure whose labels are themselves labelled φ-structures. A shape s together with a labelling $g : P(s) \to C_\varphi(X)$ that puts an *entire* φ-structure into each slot of s. This is the same data as an element of the left-hand side. Splitting each inner structure $g(p)$ into its shape $f(p)$ and its labelling gives the composite shape $\langle s, f\rangle$, and the label sitting at the composite position $(p, \hat{p})$ is the label at slot $\hat{p}$ of the inner structure at slot p.

Remark 6.10. Let $f(x) = \sum_n a_n x^n$ and $g(x) = \sum_n b_n x^n$ be the power series of ψ and φ as in Remark 6.2. The composition de-categorifies to $f(g(x))$: the number of shapes of $\psi \circ \varphi$ with n positions in total is

$$c_n = \sum_k a_k \sum_{n_1+\cdots+n_k=n} b_{n_1} b_{n_2} \cdots b_{n_k},$$

the classical formula for composing power series — an outer shape with k slots, and an inner shape in each slot, sizes summing to n.